

TEXTBOOK EDITION • PROBLEM SOLVING & ALGORITHMS

Data Structures & Algorithms (FinTech Patterns)

Arrays, Two Pointers, Sliding Window, Hashing & Algorithmic Foundations

TARGET ROLE	Developer (3+ Years Experience)
DEPARTMENT	Strategic Projects & New Age Tech, Bengaluru
PREPARED FOR	Bipin Yadav (SDE II, 4+ YoE)
ESTIMATED STUDY TIME	18 Hours
DIFFICULTY LEVEL	Intermediate to Advanced
TEACHING FORMAT	Hinglish Mental Models + Production FinTech

Table of Contents

Chapter 01 — Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)

Study Time: 35m

Chapter 02 — Array Patterns, In-Place Manipulation & Prefix Sums

Study Time: 35m

Chapter 03 — Hashing Patterns, Hash Tables & Frequency Counting

Study Time: 35m

Chapter 04 — Two Pointers Algorithmic Patterns

Study Time: 35m

Chapter 05 — Sliding Window Algorithmic Patterns

Study Time: 35m

Chapter 06 — Python for Coding Interviews & DSA Mastery

Study Time: 35m

Chapter 07 — DSA Last-Minute Algorithmic Patterns Cheatsheet

Study Time: 35m

CHAPTER 01

Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)

PRIORITY

 High Priority

DIFFICULTY

Intermediate to Advanced

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)

1. Why This Matters

In banking and financial backend systems, an algorithm that is $O(N^2)$ instead of $O(N \log N)$ can freeze a settlement engine during peak trading or UPI clearing windows. At IDFC FIRST Bank's Strategic Projects, engineers write code handling tens of thousands of concurrent payment requests. In interviews, stating an algorithm's complexity is not enough: interviewers will ask you to mathematically derive worst-case, average-case, and amortized bounds, identify auxiliary memory allocations, and analyze garbage collection overhead.

2. Prerequisites

- Basic algebra (logarithms, exponents, summation formulas: $\sum_{i=1}^n i = n(n+1)/2$).
- Understanding of stack frames and heap allocations in programming languages (Node.js/V8, Python).

3. Concept

Asymptotic Notations

- Big-O (O):** Upper bound. Formal definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that $0 \leq f(N) \leq c * g(N)$ for all $N \geq N_0$. Represents the worst-case scenario.
- Big-Omega (Ω):** Lower bound. $f(N) \geq c * g(N)$ for all $N \geq N_0$. Represents best-case performance.
- Big-Theta (Θ):** Tight bound. When an algorithm's upper bound and lower bound match: $c_1 * g(N) \leq f(N) \leq c_2 * g(N)$.
- Little-o (o):** Strict upper bound where $f(N)$ grows strictly slower than $g(N)$.

Growth of Common Time Complexities:

$O(1)$ [Constant: Hash table lookup, Array indexing]

↓

$O(\log N)$ [Logarithmic: Binary search, Balanced BST lookup]

↓

$O(N)$ [Linear: Single scan, Two Pointers, Sliding Window]

↓

$O(N \log N)$ [Linearithmic: MergeSort, QuickSort (avg), Timsort]

↓

$O(N^2)$ [Quadratic: Nested loops, BubbleSort, Brute force pairs]

↓

$O(2^N)$ [Exponential: Naive Fibonacci, Subsets recursion]

↓

$O(N!)$ [Factorial: Permutations generation, Traveling Salesperson]

Amortized Analysis

When an operation is occasionally very expensive but predominantly cheap, the *amortized cost* averages the total cost over a long sequence of operations.

The Accounting Method: You overcharge cheap operations (e.g., normal array append = 1 unit of work charged as 3 units) and store the surplus as credit. When the expensive resizing operation occurs (copying N elements = N units of work), the accumulated credit pays for the reallocation. Thus, the amortized cost per append is strictly $O(1)$.

4. Simple Example: Linear vs Logarithmic Search

```
# Linear Search:  $O(N)$  Time,  $O(1)$  Space
def linear_search(arr: list[int], target: int) → int:
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Binary Search:  $O(\log N)$  Time,  $O(1)$  Space
def binary_search(sorted_arr: list[int], target: int) → int:
    left, right = 0, len(sorted_arr) - 1
    while left ≤ right:
        mid = left + (right - left) // 2 # Prevents integer overflow
        if sorted_arr[mid] == target:
            return mid
        elif sorted_arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

5. Real-World Banking Example: High-Throughput Transaction Deduplication

Suppose an incoming UPI payment batch has $N = 100,000$ transactions. You must verify that no two transactions share the same `idempotency_key`.

- **Brute Force ($O(N^2)$):** Comparing each transaction against every other transaction requires: $(100,000 * 99,999) / 2 \approx 5 * 10^9$ operations $\rightarrow$ 5 to 10 seconds (CPU freeze and timeout).
- **Hash Set Optimization ($O(N)$ Time, $O(N)$ Space):** Checking existence in a hash set takes $O(1)$ on average: 100,000 lookups $\approx$ 0.015 seconds (15 milliseconds).
- **In-place Sort + Adjacent Scan ($O(N \log N)$ Time, $O(1)$ Auxiliary Space):** Useful when memory is constrained on embedded gateway appliances.

6. Code: Python Dynamic Array Resizing Proof

```
import sys
import time

def demonstrate_amortized_growth():
    """Demonstrates list capacity jumps in Python (CPython dynamic array)."""
    data = []
    prev_size = sys.getsizeof(data)
    print(f"{'Length':<10} | {'Byte Size':<12} | {'New Allocation Triggered?'}")
    print("-" * 50)

    for i in range(25):
        data.append(i)
        current_size = sys.getsizeof(data)
        resized = current_size != prev_size
        if resized:
            print(f"{'len(data)':<10} | {'current_size':<12} | YES (Grown to accommodate elements)")
            prev_size = current_size

if __name__ == "__main__":
    demonstrate_amortized_growth()
```

7. How It Works Internally

CPython & V8 Dynamic Array Growth Mechanics

1. In CPython, `list` is implemented as an array of pointers (`PyObject**`). When the allocated capacity is exhausted, CPython over-allocates using the formula: `new_allocated = size + (size >> 3) + (size < 9 ? 3 : 6)`. This provides an approximate geometric growth factor of $\approx 1.125\times$.
2. In V8 (Node.js/JavaScript), `JSArray` has two modes: **Fast Elements** (contiguous C++ pointer backing stores) and **Dictionary Elements** (hash tables for sparse arrays). For fast elements, resizing multiplies capacity by $1.5\times$ to $2\times$.
3. Copying N elements takes $O(N)$, but because reallocation happens geometrically, resizing from 1 to N requires: $1 + 2 + 4 + 8 + \dots + N = 2N - 1 = O(N)$ total copies across N insertions. Total Time / Operations $= O(N) / N = O(1)$ amortized per append.

8. Common Mistakes

1. **Confusing Auxiliary Space with Total Space:** Auxiliary space is temporary working memory *excluding* the input. In MergeSort, total space is $O(N)$, and auxiliary space is $O(N)$. In QuickSort, auxiliary stack space is $O(\log N)$ average, $O(N)$ worst-case.
2. **Ignoring String Immutability Overhead:** In Python and JavaScript, strings are immutable. Doing `s += char` inside an N -iteration loop creates a new string each time, turning an intended $O(N)$ operation into $O(N^2)$! Always use `"".join(list)` in Python or an array push + `.join("")` in JS.
3. **Assuming Hash Table Lookups are Always $O(1)$:** If hash collisions occur (or an adversary triggers hash collision attacks), open-addressed or chained tables degrade to $O(N)$ worst case.

9. Performance / Complexity Matrix

Operation / Algorithm	Best Time	Average Time	Worst Time	Auxiliary Space
Array Access by Index	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Dynamic Array Append	$O(1)$	$O(1)$	$O(N)$	$O(1)$
Hash Table Insert/Lookup	$O(1)$	$O(1)$	$O(N)$	$O(N)$
Binary Search	$O(1)$	$O(\log N)$	$O(\log N)$	$O(1)$
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$ avg
Breadth-First Search (BFS)	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$
Depth-First Search (DFS)	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$ stack

10. Interview Questions (Easy → Medium → Hard) with Complete Model Answers

● Easy Question

Q: What is the difference between $O(1)$ and $O(\log N)$? Give an example of each in software engineering.

Complete Answer: - **$O(1)$ [Constant Time]:** The execution time of the operation is completely independent of the input size N . No matter if the dataset contains 10 items or 10 billion items, the operation finishes in the same fixed number of machine instructions. - *Example:* Accessing an array element by index (`arr[i]`), retrieving a key from an in-memory Hash Map / Redis cache on average, or popping an element from a stack. - **$O(\log N)$ [Logarithmic Time]:** The execution time grows in proportion to the logarithm of the input size. At every step, the algorithm cuts the remaining search

space by a constant fraction (usually half). Even for $N = 1,000,000,000$ (1 billion items), $\log_2(10^9) \approx 30$ operations! – *Example:* Binary Search on a sorted array, looking up an account number in a balanced Binary Search Tree (BST) or B+ Tree index in PostgreSQL/Oracle, or finding an element in a binary heap.

● Medium Question

Q: Explain why QuickSort is commonly preferred over MergeSort for arrays in cache-sensitive systems despite QuickSort having a worse worst-case time complexity ($O(N^2)$ vs $O(N \log N)$).

Complete Answer: There are three fundamental architectural reasons: 1. **Auxiliary Memory Allocation:** – MergeSort requires an auxiliary buffer of $O(N)$ extra memory because merging two sorted subarrays cannot easily be done in-place without significant performance penalties. – QuickSort operates strictly **in-place** with only $O(\log N)$ auxiliary call-stack memory. 2. **CPU Cache Locality & Hardware Prefetching:** – QuickSort's partitioning traverses memory **sequentially and contiguously** from both ends inward (`left` and `right` pointers). This exhibits exceptional **spatial locality of reference**. – Modern CPUs load entire 64-byte cache lines into L1/L2 caches. Because QuickSort accesses adjacent bytes, almost every read is an ultra-fast L1 cache hit (~1–2 ns) rather than a slow main RAM fetch (~60–100 ns). – MergeSort frequently copies elements back and forth between the primary array and auxiliary buffer, saturating CPU memory bus bandwidth and thrashing cache lines. 3. **Randomized / Dual-Pivot Mitigations:** – Modern implementations (such as Java's `Arrays.sort` for primitives) use **Dual-Pivot QuickSort** with randomized or median-of-three pivot selection. This makes the probability of encountering the pathological $O(N^2)$ worst-case virtually zero (1 in 2^{32}).

When is MergeSort preferred? – When **Stability** is non-negotiable (preserving the relative order of duplicate elements). – When sorting linked lists (where nodes are linked by pointers and no contiguous memory copying is required). – When sorting external files stored on disk that do not fit into RAM (External Merge Sort).

● Hard Question

Q: An algorithm processes a stream of financial transactions. Each incoming item requires checking a min-heap of size K and occasionally rebuilding an auxiliary index of size M . Derive the tight asymptotic bound for N total transactions where $M = \sqrt{N}$ and $K = \log N$.

Complete Mathematical Derivation: Let us break down the total work into two components: 1. **Heap Operation Cost (per transaction):** – For each of the N transactions, inserting/checking a min-heap of size K takes $O(\log K)$ time. – Given $K = \log N$: `Cost_heap =  $O(\log(\log N))$` per transaction. – Total heap cost across all N transactions: `Total_heap =  $N * O(\log(\log N)) = O(N \log(\log N))$` .

1. Index Rebuilding Cost:

2. An auxiliary index of size $M = \sqrt{N}$ is rebuilt periodically.

3. Rebuilding an index of size M takes `$O(M \log M)$` time (e.g., sorting or constructing a balanced tree).

4. If the index is rebuilt every M transactions, the total number of rebuilds across N transactions is:

$$\text{Number of Rebuilds} = N / M = N / \sqrt{N} = \sqrt{N}.$$

5. Work per rebuild: `$M \log M = \sqrt{N} * \log(\sqrt{N}) = \sqrt{N} * (1/2 * \log N) = (1/2) * \sqrt{N} \log N$` .

6. Total rebuild work across all $\sqrt{N}$ rebuilds: $\text{Total_rebuild} = \sqrt{N} * [(1/2) * \sqrt{N} \log N] = (1/2) * (\sqrt{N} * \sqrt{N}) \log N = (1/2) * N \log N = O(N \log N)$.

7. Combined Tight Bound:

8. $\text{Total Time} = \text{Total_heap} + \text{Total_rebuild} = O(N \log(\log N)) + O(N \log N)$.

9. Since $N \log N$ strictly dominates $N \log(\log N)$ as N grows large: $\text{Final Tight Bound} = \theta(N \log N)$.

11. Follow-up Questions from Interviewer with Complete Answers

Follow-up 1

Interviewer: "What is cache locality, and how does it affect the empirical execution speed of an $O(N)$ array traversal versus an $O(N)$ linked list traversal?"

Complete Answer: - **Cache Locality Principles:** - **Spatial Locality:** If a memory location is accessed, nearby memory locations are likely to be accessed soon. - **Temporal Locality:** If a memory location is accessed, the same location is likely to be accessed again in the near future. - **Why Arrays Beat Linked Lists by 10x-50x Empirically:** - An **array** is allocated in a single, contiguous block of virtual memory. When the CPU reads `arr[0]`, the hardware prefetcher automatically pulls the entire 64-byte CPU cache line (holding the next 8 to 16 integer elements) into L1 cache. Subsequent iterations (`arr[1]`, `arr[2]`, etc.) are instant L1 hits. - In a **linked list**, each node is allocated independently on the heap via dynamic memory allocators (`malloc` / garbage collector). Nodes are scattered unpredictably across RAM. - To traverse a linked list, the CPU must dereference `node.next` (known as **pointer chasing**). Because the next node is at an arbitrary memory address, the CPU experiences a **cache miss**, stalling the execution pipeline for 100-200 clock cycles while waiting for data from main memory. - **Conclusion:** While both algorithms are asymptotically $O(N)$, the array traversal completes in a fraction of the wall-clock time due to spatial cache locality and hardware prefetching.

Follow-up 2

Interviewer: "If `Array.prototype.sort()` in Node.js uses Timsort, what is its worst-case space complexity, and why doesn't it use standard in-place QuickSort?"

Complete Answer: 1. **Worst-Case Space Complexity of Timsort:** - Timsort requires **$O(N)$** auxiliary space in the worst case (or $O(N/2)$ buffer memory) to merge runs of sorted elements, alongside a small stack of $O(\log N)$ run descriptors. 2. **Why Node.js (V8) Replaced QuickSort with Timsort (starting in V8 v7.0):** - **Stability is Essential for Web Applications:** QuickSort is unstable (equal elements can change their relative order). For example, if a user sorts a banking transaction table by "Transaction Date" and then sorts by "Amount", an unstable sort scrambles the date ordering of equal-amount transactions! Timsort guarantees strict **stability**. - **Real-World Data is Already Partially Sorted:** Timsort is an adaptive algorithm derived from MergeSort and InsertionSort. On already sorted or reversed data (very common in logging and audit trails), Timsort detects natural runs and runs in **$O(N)$ best-case linear time!** QuickSort would still take $O(N \log N)$ or degrade to $O(N^2)$ without complex pivot balancing. -

Guaranteed $O(N \log N)$ Worst-Case: Unlike naive QuickSort which can degrade to $O(N^2)$ when encountering adversarial worst-case inputs (causing Node.js event loop freezes), Timsort strictly guarantees $O(N \log N)$ worst-case time.

Follow-up 3

Interviewer: "How does Python's list over-allocation strategy affect memory footprint when scaling to millions of ledger rows?"

Complete Answer:

- Pointer Indirection Overhead:** - In Python (CPython), a `list` does not store primitive numbers directly. It is an array of 8-byte C pointers (`PyObject*`), where each pointer references an independent heap object (`PyLongObject`). - A 64-bit Python integer object contains: - Reference count: 8 bytes - Type pointer (`ob_type`): 8 bytes - Size field: 8 bytes - Digit array: 4-8 bytes - **Total per integer object:** 28 bytes! - Plus the 8-byte pointer in the list = **36 bytes per integer element**.
- Geometric Over-Allocation:** - When elements are appended, CPython allocates extra unused slots: `new_allocated = size + (size >> 3) + (size < 9 ? 3 : 6)`. - This means at any point, a list may hold 12.5% more allocated capacity than its current `len()`.
- Scale Comparison (10 Million Transactions):** - Raw Python List of integers: `10,000,000 * 36 bytes ≈ 360 MB`. - NumPy Array (`np.int64`) or C array: `10,000,000 * 8 bytes ≈ 80 MB` (contiguous, zero pointer overhead, 4.5x memory reduction).

Production Recommendation: For financial transaction processing pipelines exceeding 100,000 records, always migrate from raw Python lists to NumPy arrays, pandas DataFrames, or PyArrow tables to prevent memory exhaustion and garbage collection thrashing.

Follow-up 4

Interviewer: "How do you analyze the amortized cost of an append operation using both the Aggregate Method and the Accounting / Potential Method?"

Complete Answer:

1. The Aggregate Method

- In a sequence of N append operations starting from an empty array of capacity 1:
- Resizing occurs at sizes: 1, 2, 4, 8, 16, ..., 2^k (where $2^k \leq N$).
- The actual cost of insertion without resize is 1 for each of the N operations $\rightarrow$ Total = N .
- The copying cost during resizes is: $1 + 2 + 4 + 8 + \dots + 2^k < 2 * 2^k \leq 2N$.
- Total Cost across N operations = N (insertions) + $2N$ (copies) = $3N$.
- **Amortized cost per operation:** Total Cost / $N = 3N / N = 3 = O(1)$.

2. The Accounting Method (Token / Banker's Method)

- We assign an artificial **amortized charge** of 3 credits to each incoming append operation:
- **Credit 1:** Pays for the actual insertion of the new element into the current slot.
- **Credit 2:** Stored as savings on this new element to pay for its own future relocation.
- **Credit 3:** Stored as savings on an older element that has already moved once but needs to move again.

- When the array doubles from size K to $2K$, exactly K new elements have arrived since the last resize.
- Accumulated credit = $K * 2 = 2K$ credits!
- This accumulated credit is precisely enough to copy all $2K$ elements into the newly allocated memory buffer without charging any extra cost.
- Because the credit balance never drops below zero, the amortized cost is strictly $O(1)$.

3. The Potential Method (Physicist's Method)

- Define a potential function $\Phi(D_i)$ on the data structure state after step i : $\Phi(D_i) = 2 * \text{size}_i - \text{capacity}_i$
- Immediately after a doubling resize: $\text{size} = \text{capacity} / 2 \rightarrow \Phi = 2 * (\text{capacity} / 2) - \text{capacity} = 0$.
- Immediately before a doubling resize: $\text{size} = \text{capacity} \rightarrow \Phi = 2 * \text{capacity} - \text{capacity} = \text{capacity}$.
- Since $\Phi(D_i) \geq 0$ for all i , the potential is always non-negative.
- The amortized cost is defined as: $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.
- **Case 1: No resize ($c_i = 1$):** $\text{size}_i = \text{size}_{i-1} + 1$, $\text{capacity}_i = \text{capacity}_{i-1}$. $\Delta\Phi = (2(\text{size} + 1) - \text{capacity}) - (2 * \text{size} - \text{capacity}) = 2$. $\hat{c}_i = 1 + 2 = 3$.
- **Case 2: Resize occurs ($c_i = \text{size}_{i-1} + 1$):** $\text{capacity}_i = 2 * \text{capacity}_{i-1} = 2 * \text{size}_{i-1}$. $\Phi(D_{i-1}) = \text{size}_{i-1}$. $\Phi(D_i) = 2 * (\text{size}_{i-1} + 1) - 2 * \text{size}_{i-1} = 2$. $\Delta\Phi = 2 - \text{size}_{i-1}$. $\hat{c}_i = (\text{size}_{i-1} + 1) + (2 - \text{size}_{i-1}) = 3$.
- In both cases, the amortized cost is strictly $3 = O(1)$.

12. Practical Exercise

Analyze the time and auxiliary space complexity of each function:

```
# Exercise 1:
def mystery_one(n: int):
    count = 0
    i = n
    while i > 0:
        for j in range(i):
            count += 1
        i /= 2
    return count

# Exercise 2:
def mystery_two(arr: list[int]):
    n = len(arr)
    result = []
    for i in range(n):
        subset = []
        for j in range(i, n):
            subset.append(arr[j])
            result.append(list(subset))
    return result
```

Complete Mathematical Solutions:

Solution for Mystery 1:

- The outer loop divides `i` by 2 at each iteration (`n, n/2, n/4, n/8, ..., 1`).
- The inner loop executes `i` times.
- Total iterations: $T(N) = N + N/2 + N/4 + N/8 + \dots + 1$ $T(N) = N * (1 + 1/2 + 1/4 + 1/8 + \dots)$ This is a converging geometric series with sum $= N * [1 / (1 - 1/2)] = 2N$.
- **Time Complexity: $O(N)$** (Linear, NOT $O(N \log N)$!).
- **Auxiliary Space Complexity: $O(1)$** (Only scalar counter variables).

Solution for Mystery 2:

- The outer loop runs N times. The inner loop runs from `i` to N .
- Total iterations $= N + (N - 1) + (N - 2) + \dots + 1 = N(N + 1) / 2 = O(N^2)$.
- However, inside the loop, `result.append(list(subset))` creates a shallow copy of the current `subset`.
- The average length of `subset` is $O(N)$. Copying an array of length K takes $O(K)$ time!
- Total time $= \Sigma(\text{iterations}) * \text{copy_length} = O(N^2 * N) = O(N^3)$.
- **Time Complexity: $O(N^3)$** .
- **Auxiliary Space Complexity: $O(N^3)$** (to store all generated subarrays in the result list).

13. Quick Revision

- Big- O denotes worst-case upper bound; Big- Θ denotes tight asymptotic bound; Big- Ω denotes lower bound.
- Geometric series: $N + N/2 + N/4 + \dots = 2N = O(N)$.
- In-place algorithms utilize $O(1)$ auxiliary space beyond input buffers.
- String concatenation inside loops in Python/JS creates $O(N^2)$ memory churning due to immutability.
- Dynamic array append is $O(1)$ amortized, $O(N)$ worst-case.

14. Interview Checklist

- [x] Can define Big- O mathematically ($0 \leq f(N) \leq c * g(N)$).
- [x] Can explain the accounting/potential method for amortized analysis.
- [x] Can distinguish auxiliary space from input space.
- [x] Can explain CPU cache lines and spatial/temporal locality.
- [x] Can identify hidden $O(N)$ costs in high-level language methods (e.g., `list.insert(0, val)` or `Array.shift()` which shifts all elements).
- [x] Can mathematically derive complex series summation bounds on whiteboard.



CHAPTER 01 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Basic algebra (logarithms, exponents, summation formulas: $\sum_{i=1}^n i = n(n+1)/2$).
- Understanding of stack frames and heap allocations in programming languages (Node.js/V8, Python).
- Big-O (O): Upper bound. Formal definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that $0 \leq f(N)$



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 02

Array Patterns, In-Place Manipulation & Prefix Sums

PRIORITY

● High Priority

DIFFICULTY

Intermediate to Advanced

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Array Patterns, In-Place Manipulation & Prefix Sums

1. Why This Matters

Arrays form the backbone of memory management and algorithmic screening at IDFC FIRST Bank. In live coding interviews, array problems test your ability to avoid auxiliary allocations, identify prefix accumulations, manipulate boundaries without off-by-one errors, and optimize brute-force $O(N^2)$ loops into $O(N)$ linear scans.

2. Prerequisites

- Familiarity with zero-indexed array indexing and contiguous memory buffers.
- Understanding of time vs space complexity trade-offs.

3. Concept

Key Array Algorithmic Patterns

1. **Prefix Sum (Cumulative Sum):** Precomputing running totals allows any range sum query `sum(arr[i...j])` to be answered in $O(1)$ time via `prefix[j+1] - prefix[i]`.
2. **In-Place Two Pointers / Swapping:** Modifying array values without creating auxiliary lists, maintaining $O(1)$ extra space.
3. **Dutch National Flag (Three-Way Partitioning):** Partitioning an array around pivot values in a single linear pass.

```
flowchart LR
    subgraph PrefixSum["Prefix Sum Array Construction"]
        arr["arr: [ 10, 20, 15, 30 ]"]
        pre["pref: [ 0, 10, 30, 45, 75 ]"]
    end
    end
    arr --> pre
```

$\text{RangeSum}(i, j) = \text{pref}[j+1] - \text{pref}[i]$

4. Simple Example: In-Place Removal of Duplicates

```
def remove_duplicates_sorted(nums: list[int]) → int:
    """
    Removes duplicates in-place from a sorted array.
    Returns the count of unique elements.
    Complexity: O(N) Time, O(1) Auxiliary Space.
    """
    if not nums:
        return 0

    write_ptr = 1
    for read_ptr in range(1, len(nums)):
        if nums[read_ptr] != nums[read_ptr - 1]:
            nums[write_ptr] = nums[read_ptr]
            write_ptr += 1

    return write_ptr
```

5. Real-World Banking Example: Daily Transaction Range Sum Engine

A core banking ledger receives thousands of audit requests: "What was the total volume transacted between timestamp T_1 and T_2 ?" - Re-summing transactions on each query is $O(N)$ per request, overloading the database under high concurrency. - By maintaining an hourly or daily **Prefix Sum Ledger**, the balance query computes in $O(1)$ instant time via a simple subtraction.

6. Code: Product of Array Except Self ($O(N)$ Time, $O(1)$ Extra Space)

Problem Statement

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`. You must solve it **without using the division operation** and in $O(N)$ time.

Clarifying Questions

1. *Can elements be zero or negative?* Yes, zeroes are explicitly allowed.
2. *Can the product overflow a standard 32-bit integer?* Yes, in Python integers have arbitrary precision, but in languages like Java/C++, 64-bit integers (`long`) must be used.
3. *Does the output array count toward auxiliary space?* No, output space is excluded from auxiliary space analysis.

Brute-Force Approach

For every index i , iterate through the rest of the array with a nested loop and calculate the product. -

Time Complexity: $O(N^2)$ - **Space Complexity:** $O(1)$

Optimized Approach: Prefix and Suffix Running Products

Instead of recomputing products repeatedly, notice that: product except i = (product of elements to the left of i) * (product of elements to the right of i)

```
def product_except_self(nums: list[int]) → list[int]:
    n = len(nums)
    output = [1] * n

    # Step 1: Compute prefix products in-place in output array
    prefix = 1
    for i in range(n):
        output[i] = prefix
        prefix *= nums[i]

    # Step 2: Traverse backwards and multiply by running suffix product
    suffix = 1
    for i in range(n - 1, -1, -1):
        output[i] *= suffix
        suffix *= nums[i]

    return output

# Test
print(product_except_self([1, 2, 3, 4])) # [24, 12, 8, 6]
print(product_except_self([-1, 1, 0, -3, 3])) # [0, 0, 9, 0, 0]
```

- **Time Complexity:** $O(N)$ (Two passes of length N).
- **Space Complexity:** $O(1)$ auxiliary space (only scalar `prefix` and `suffix` trackers).
- **Edge Cases:** Array with a single zero, multiple zeroes (where all outputs become zero), all negative numbers.

7. How It Works Internally: Memory Layout & Cache Locality

Arrays in RAM are stored in a contiguous block of virtual memory. When the CPU executes `output[i] = prefix`, hardware prefetchers load a **cache line** (typically 64 bytes) into L1/L2 cache. – Forward and backward sequential scans have **100% spatial locality**, meaning almost zero cache misses. – If this were implemented using a linked list or pointer-heavy tree, each node access would require an unpredictable memory hop, causing severe cache line stalls.

8. Common Mistakes

1. **Division by Zero:** Attempting to solve *Product of Array Except Self* by calculating the total product and dividing by `nums[i]` crashes when `nums[i] == 0`.
2. **Off-by-One in Prefix Sums:** Creating a prefix array of size N instead of $N+1$. Using size $N+1$ with `prefix[0] = 0` eliminates messy `if i == 0:` boundary checks.
3. **Modifying Array While Iterating:** Mutating an array with `.pop()` or `.insert()` during a `for x in arr:` loop alters the iteration index, leading to skipped elements.

9. Performance / Complexity Matrix

Problem / Technique	Brute Force	Optimized Time	Auxiliary Space
Prefix Sum Range Query	$O(N)$ per query	$O(1)$ per query ($O(N)$ precompute)	$O(N)$
Product Except Self	$O(N^2)$	$O(N)$	$O(1)$
Remove Duplicates In-Place	$O(N^2)$	$O(N)$	$O(1)$
Rotate Array by K steps	$O(N * K)$	$O(N)$ (Triple reverse)	$O(1)$

10. Interview Questions (Easy → Medium → Hard)

● Easy Question: In-Place Array Reversal

Q: How do you reverse an array in-place without allocating any extra auxiliary memory?

Complete Answer: Use a two-pointer converging approach. Initialize a `left` pointer at index 0 and a `right` pointer at index `len(arr) - 1`. While `left < right`, swap `arr[left]` and `arr[right]`, then increment `left` and decrement `right`.

```
def reverse_array_inplace(arr: List[int]) → None:
    """Reverses an array in-place.
    Time Complexity:  $O(N)$  -  $N/2$  swaps.
    Auxiliary Space:  $O(1)$  - purely scalar pointer manipulation.
    """
    left, right = 0, len(arr) - 1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1

# Verification
nums = [10, 20, 30, 40, 50]
reverse_array_inplace(nums)
assert nums == [50, 40, 30, 20, 10]
```

● Medium Question: Maximum Subarray Sum (Kadane's Algorithm)

Q: Given an unsorted array of integers containing positive and negative numbers, find the contiguous subarray which has the largest sum. Extend this to return the starting and ending indices of the subarray.

Complete Answer: Kadane's algorithm uses dynamic programming with state reduction: - At each index `i`, we decide whether to extend the previous subarray (`current_max + nums[i]`) or start fresh from `nums[i]`. - If `current_max` drops below zero, keeping it hurts future sums; we reset and start fresh. - To track indices, update `start_idx` whenever a new subarray starts.

```
def max_sub_array_with_indices(nums: list[int]) → tuple[int, int, int]:
    """Returns (max_sum, start_index, end_index).
    Time Complexity: O(N) single pass.
    Auxiliary Space: O(1).
    """
    global_max = nums[0]
    current_max = nums[0]
    start = end = temp_start = 0

    for i in range(1, len(nums)):
        if nums[i] > current_max + nums[i]:
            current_max = nums[i]
            temp_start = i
        else:
            current_max += nums[i]

        if current_max > global_max:
            global_max = current_max
            start = temp_start
            end = i

    return global_max, start, end

# Verification
max_sum, s, e = max_sub_array_with_indices([-2, 1, -3, 4, -1, 2, 1, -5, 4])
assert (max_sum, s, e) == (6, 3, 6) # Subarray [4, -1, 2, 1] sum = 6
```

● Hard Question: Best Time to Buy and Sell Stock III (At Most 2 Transactions)

Q: Given an array representing daily stock prices, find the maximum profit you can achieve with at most **two transactions**. You may not engage in multiple transactions concurrently (you must sell before you buy again).

Complete Answer: Solve this in **O(N) time and O(1) auxiliary space** using a 4-state dynamic programming state machine: - **buy1**: The effective cost of the first purchase (minimize cash spent: $\min(\text{buy1}, \text{price})$). - **sell1**: The maximum profit after the first sale ($\max(\text{sell1}, \text{price} - \text{buy1})$). - **buy2**: The effective cost of the second purchase reinvesting first-sale profit ($\min(\text{buy2}, \text{price} - \text{sell1})$). - **sell2**: The maximum cumulative profit after the second sale ($\max(\text{sell2}, \text{price} - \text{buy2})$).

```
def max_profit_two_transactions(prices: list[int]) → int:
    """Computes max profit with at most two buy-sell transactions.
    Time Complexity: O(N) single pass.
    Auxiliary Space: O(1) across 4 scalar state registers.
    """
    if not prices:
        return 0

    buy1 = buy2 = float('inf')
    sell1 = sell2 = 0

    for price in prices:
        buy1 = min(buy1, price)
        sell1 = max(sell1, price - buy1)
        buy2 = min(buy2, price - sell1) # Reinvest profit from transaction 1
        sell2 = max(sell2, price - buy2)

    return sell2

# Verification
assert max_profit_two_transactions([3, 3, 5, 0, 0, 3, 1, 4]) == 6 # Buy at 0, sell at 3; buy at 1, sell at 4
assert max_profit_two_transactions([1, 2, 3, 4, 5]) == 4 # Single transaction buy at 1, sell at 5
assert max_profit_two_transactions([7, 6, 4, 3, 1]) == 0 # Downward trend, 0 profit
```

11. Follow-up Questions from Interviewer with Complete Answers

Follow-up 1

Interviewer: "In Kadane's algorithm, what happens if all numbers in the array are negative? How do you ensure your code doesn't mistakenly return 0?"

Complete Answer: - **The Bug:** If you initialize `global_max = 0` and `current_max = 0`, then for an array of all negative numbers like `[-5, -2, -9]`, the algorithm will discard every negative element and return `0`. But `0` corresponds to an empty subarray, which violates the constraint of returning a non-empty contiguous subarray. - **The Mathematical Fix:** Initialize both `global_max` and `current_max` strictly to `nums[0]`. In the loop, iterate from index 1. - **Why It Works:** - For `[-5, -2, -9]`: - Index 0: `current_max = -5`, `global_max = -5`. - Index 1: `current_max = max(-2, -5 + (-2)) = max(-2, -7) = -2`. `global_max = max(-5, -2) = -2`. - Index 2: `current_max = max(-9, -2 + (-9)) = max(-9, -11) = -9`. `global_max = max(-2, -9) = -2`. - The algorithm mathematically returns `-2` (the least negative element), which is the exact optimal single-element subarray.

Follow-up 2

Interviewer: "Can you modify your range sum query to handle dynamic in-place updates in $O(\log N)$ time? Why doesn't a standard prefix sum array work?"

Complete Answer: - **Why Prefix Sum Array Fails for Dynamic Updates:** - A static prefix sum array computes range sums in $O(1)$ time. - However, when a single element `arr[k]` is updated, all subsequent prefix values `prefix[k+1] ... prefix[N]` must be recomputed! This takes **$O(N)$ time per point update**. In a high-frequency banking trading ledger with 50,000 updates/sec, $O(N)$ updates will

cause total system collapse. - **The Optimal Solution: Binary Indexed Tree (Fenwick Tree):** - A Fenwick Tree maintains partial cumulative sums using powers of 2 via the bitwise low-bit formula: `lowbit(i) = i & (-i)`. - **Point Update** `arr[k] += delta`: Takes $O(\log N)$ time by traversing up the tree: `idx += idx & (-idx)`. - **Prefix Query** `sum(1...k)`: Takes $O(\log N)$ time by peeling off the lowest set bits: `idx -= idx & (-idx)`. - **Range Query** `sum(L...R)`: `query(R) - query(L - 1)` in $O(\log N)$ time. - **Auxiliary Space:** $O(N)$.

```
class FenwickTree:
    """Binary Indexed Tree supporting  $O(\log N)$  point updates and  $O(\log N)$  range queries."""
    def __init__(self, size: int):
        self.size = size
        self.tree = [0] * (size + 1) # 1-indexed

    def update(self, index: int, delta: int) → None:
        """Adds delta to element at 1-based index in  $O(\log N)$  time."""
        while index ≤ self.size:
            self.tree[index] += delta
            index += index & (-index) # Add lowest set bit

    def query_prefix(self, index: int) → int:
        """Returns cumulative sum from 1 to index in  $O(\log N)$  time."""
        total = 0
        while index > 0:
            total += self.tree[index]
            index -= index & (-index) # Strip lowest set bit
        return total

    def query_range(self, left: int, right: int) → int:
        """Returns range sum from left to right in  $O(\log N)$  time."""
        return self.query_prefix(right) - self.query_prefix(left - 1)

# Verification: Banking Balance Ledger with dynamic transactions
ft = FenwickTree(5)
initial_balances = [100, 250, 75, 400, 50]
for idx, bal in enumerate(initial_balances, 1):
    ft.update(idx, bal)

# Query range accounts 2 to 4: 250 + 75 + 400 = 725
assert ft.query_range(2, 4) == 725

# Dynamic deposit: Account 3 receives +125 INR in  $O(\log N)$ 
ft.update(3, 125)
assert ft.query_range(2, 4) == 850 # Instant updated range sum
```

13. Practical Exercise

Implement the **Rotate Array by K Steps** in-place in $O(N)$ time and $O(1)$ space using the 3-step array reversal technique: 1. Reverse entire array. 2. Reverse first K elements. 3. Reverse remaining $N - K$ elements.

(Ensure $K = K \bmod N$ handles cases where $K \geq N$).

14. Quick Revision

- Prefix sum: $\text{sum}(i, j) = \text{pref}[j+1] - \text{pref}[i]$.

- In-place two pointers: read pointer scans ahead, write pointer retains valid output.
- Kadane's: $\text{local} = \max(\text{num}, \text{local} + \text{num})$.
- In-place reversal eliminates auxiliary space for rotation problems.

15. Interview Checklist

- ☐ Writes Kadane's algorithm correctly for all-negative inputs.
- ☐ Solves Product Except Self without division.
- ☐ Understands why $N+1$ sized prefix arrays prevent boundary condition bugs.
- ☐ Explains cache line prefetching benefits of contiguous memory.

CHAPTER 02 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Familiarity with zero-indexed array indexing and contiguous memory buffers.
- Understanding of time vs space complexity trade-offs.
- Re-summing transactions on each query is $O(N)$ per request, overloading the database under high concurrency.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 03

Hashing Patterns, Hash Tables & Frequency Counting

PRIORITY

● High Priority

DIFFICULTY

Intermediate to Advanced

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Hashing Patterns, Hash Tables & Frequency Counting

1. Why This Matters

Hash tables are the most ubiquitous data structure in modern backend engineering. At IDFC FIRST Bank, hash maps power distributed session caching, idempotency tracking, fraud velocity checks, and fast in-memory record matching. In coding interviews, recognizing that an $O(N^2)$ brute-force lookup can be converted into an $O(N)$ linear time algorithm using a hash map or hash set is the single most frequent optimization pattern.

2. Prerequisites

- Understanding of key-value associations.
- Basic knowledge of hash functions and bucket indexing.

3. Concept

Hash Map Fundamentals

A hash table maps keys to values by passing the key through a **hash function** to compute an integer hash code, which is then mapped to an internal bucket index using modulo arithmetic: $\text{bucket_index} = \text{hash}(\text{key}) \bmod \text{capacity}$

```
flowchart LR
    subgraph Hashing["Hash Function & Bucket Resolution"]
        k1["Key: 'pan_ABCDE1234F'"] --> h["Hash Function"]
        h --> code["Hash: 0x8F3A29"]
        code --> mod["Modulo Capacity (16)"]
        mod --> b9["Bucket #9"]
    end
```

Collision Resolution Strategies

1. **Separate Chaining:** Each bucket stores a linked list (or balanced red-black tree, as in Java 8+ `HashMap`) of key-value pairs that hash to the same bucket.

2. **Open Addressing (Linear/Quadratic/Pseudo-random Probing)**: All entries reside directly within the bucket array. If a collision occurs, the algorithm systematically probes adjacent or pseudo-random indices until an empty slot is found (used by Python dictionaries).

4. Simple Example: Two Sum (O(N) Time, O(N) Space)

```
def two_sum(nums: list[int], target: int) → list[int]:
    """
    Finds indices of two numbers that add up to target.
    Complexity: O(N) Time, O(N) Space.
    """
    seen = {} # value → index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

5. Real-World Banking Example: Real-Time Fraud Velocity Detection

During a flash sale, card fraud detection rules require checking: *"Has this card transacted more than 3 times in the past 60 seconds?"* - Scanning the relational transactions table each time would overwhelm the database with disk I/O. - By maintaining an in-memory hash table (or Redis sorted set / hash), the system can record timestamped attempts by `card_id` and check velocity thresholds in sub-millisecond O(1) time.

6. Code: Subarray Sum Equals K (Prefix Sum + Hash Map)

Problem Statement

Given an array of integers `nums` and an integer `k`, return the total number of continuous subarrays whose sum equals to `k`.

Clarifying Questions

1. *Can elements be negative?* Yes! Because numbers can be negative, a sliding window **cannot** be used because the running sum does not increase monotonically.
2. *Can the array be empty?* Assume $1 \leq \text{len} \leq 20,000$.

Intuition: Prefix Sum Complement

Let $\text{pref}[i]$ be the sum of elements from index 0 to i . A subarray from j to i has sum equal to k if and only if:
 $\text{pref}[i] - \text{pref}[j-1] = k$ iff $\text{pref}[j-1] = \text{pref}[i] - k$

By storing the frequency of all previous prefix sums in a hash map, we can instantly look up how many times $\text{pref}[i] - k$ has appeared.

```

from collections import defaultdict

def subarray_sum_k(nums: list[int], k: int) → int:
    prefix_counts = defaultdict(int)
    prefix_counts[0] = 1 # Base case: a prefix sum of 0 has occurred once

    running_sum = 0
    total_subarrays = 0

    for num in nums:
        running_sum += num
        # Check how many prior prefixes satisfy: running_sum - prior_prefix = k
        complement = running_sum - k
        if complement in prefix_counts:
            total_subarrays += prefix_counts[complement]

        prefix_counts[running_sum] += 1

    return total_subarrays

# Test cases
print(subarray_sum_k([1, 1, 1], 2))          # 2 ([1,1] at index 0..1 and 1..2)
print(subarray_sum_k([1, -1, 0], 0))         # 3 ([1,-1], [0], [1,-1,0])
print(subarray_sum_k([3, 4, 7, 2, -3, 1, 4, 2], 7)) # 4

```

- **Time Complexity:** $O(N)$ single pass.
- **Space Complexity:** $O(N)$ to store prefix frequencies in the hash map.
- **Edge Cases:** Array with zeroes, negative numbers, $k = 0$, single element matching k .

7. How It Works Internally: The Hash Collision Vulnerability (HashDoS)

If an attacker knows the hashing algorithm used by an application server (e.g., standard SipHash or MurmurHash without a random seed), they can send thousands of HTTP POST parameters designed to produce identical bucket indices. - When thousands of keys collide in the same bucket, lookup complexity degrades from $O(1)$ to $O(N)$. - The server CPU spikes to 100% parsing a single malicious request. Modern runtimes (Node.js/V8, Python 3.4+) randomize the hash seed (`PYTHONHASHSEED`) on startup to prevent HashDoS attacks.

8. Common Mistakes

1. **Forgetting the Base Case** `prefix_counts[0] = 1` : In prefix sum hashing problems, omitting `prefix_counts[0] = 1` causes the algorithm to miss any valid subarray that begins at index 0!
2. **Using Mutable Keys in Hash Maps:** Keys in a hash map must be immutable (hashable). In Python, lists cannot be keys (`TypeError: unhashable type: 'list'`); use a `tuple` instead.
3. **Assuming Hash Maps are Ordered Across All Languages:** While Python 3.7+ and JavaScript maintain insertion order, standard hash maps in C++ (`std::unordered_map`) or Go maps do not guarantee order. Never rely on order unless language-guaranteed.

9. Performance / Complexity Matrix

Problem / Technique	Brute Force Time	Hash Map Optimized Time	Auxiliary Space
Two Sum	$O(N^2)$	$O(N)$	$O(N)$
Subarray Sum Equals K	$O(N^2)$	$O(N)$	$O(N)$
Group Anagrams	$O(N * K \log K)$	$O(N * K)$ (Character count key)	$O(N * K)$
Longest Consecutive Sequence	$O(N \log N)$ (Sort)	$O(N)$ (Hash Set)	$O(N)$

● Easy Question: Hash Set vs Hash Map Internals

Q: How does a Hash Set differ from a Hash Map under the hood in Python and Java, and why can't a mutable list be used as a key?

Complete Answer: 1. **Under the Hood Architecture:** - In Java, `HashSet<E>` is literally a wrapper around an internal `HashMap<E, Object>` instance where every element `e` is inserted as `map.put(e, DUMMY_OBJECT)`. - In Python, `set` and `dict` share nearly identical CPython internal hash table implementations (open addressing with quadratic/perturbation probing), but a `set` stores only `setentry` structs containing `{hash, key}` without allocating space for value pointers, saving approximately 33% memory compared to `dict`. 2. **Why Mutable Keys Corrupt the Hash Table:** - When a key is inserted, its bucket index is computed as: `bucket = hash(key) mod capacity`. - If the key were mutable (like a Python `list`) and modified after insertion, its `hash(key)` would change! - When you subsequently call `key in map`, the engine recomputes `hash(key) mod capacity`, lands on a completely different bucket, and returns `False` or raises an internal exception. The original entry is now permanently orphaned in memory. - Therefore, languages enforce that keys must implement an immutable `__hash__()` and `__eq__()` protocol.

● Medium Question: Longest Consecutive Sequence in $O(N)$ Time

Q: Given an unsorted array of integers, find the length of the longest consecutive elements sequence in strictly $O(N)$ time without sorting.

Complete Answer: Sorting takes $O(N \log N)$, so we must use a Hash Set for $O(1)$ lookups. 1. Insert all numbers into a Python `set`. 2. A number `x` is the start of a streak if and only if `x - 1` is NOT in the set. 3. If `x - 1 in num_set`, skip it immediately (it will be counted from its true origin). 4. If `x - 1 not in num_set`, run a `while` loop checking `x + 1, x + 2, ...`. 5. Each number is visited at most twice. Total time is strictly $O(N)$.

```
def longest_consecutive(nums: list[int]) → int:
    """Finds longest consecutive elements sequence in O(N) time.
    Auxiliary Space: O(N) for the hash set.
    """
    num_set = set(nums)
    longest_streak = 0

    for num in num_set:
        # Only start counting if num is the beginning of a streak
        if num - 1 not in num_set:
            current_num = num
            current_streak = 1

            while current_num + 1 in num_set:
                current_num += 1
                current_streak += 1

            longest_streak = max(longest_streak, current_streak)

    return longest_streak

# Verification
assert longest_consecutive([100, 4, 200, 1, 3, 2]) == 4 # [1, 2, 3, 4]
assert longest_consecutive([0, 3, 7, 2, 5, 8, 4, 6, 0, 1]) == 9
assert longest_consecutive([]) == 0
```

● Hard Question: Design an O(1) Key-Value Store with `getRandomKey()`

Q: Design a data structure supporting `insert(val)`, `remove(val)`, and `getRandom()` all executing in strictly **O(1) average time**.

Complete Answer: - **The Challenge:** - A Hash Map provides O(1) `insert` and `remove`, but picking a random element uniformly requires O(N) iteration or random bucket sampling with non-deterministic tombstones. - An Array provides O(1) `getRandom` via `arr[random.randint(0, len - 1)]`, but removing an arbitrary element takes O(N) due to element shifting. - **The Hybrid Solution (Map + Array with Swap-to-End Deletion):** - Use a dynamic array `self.values` to store items contiguously. - Use a hash map `self.indices` mapping `val → array_index`. - **O(1) Deletion:** 1. Look up `idx = self.indices[val]`. 2. Swap `self.values[idx]` with `self.values[-1]` (the last element). 3. Update `self.indices[last_val] = idx`. 4. Call `self.values.pop()` and `del self.indices[val]`. Both operations are strictly O(1)!

```

import random

class RandomizedSet:
    """Data structure with O(1) insert, remove, and uniform getRandom."""
    def __init__(self):
        self.indices: dict[int, int] = {}
        self.values: list[int] = []

    def insert(self, val: int) → bool:
        if val in self.indices:
            return False
        self.indices[val] = len(self.values)
        self.values.append(val)
        return True

    def remove(self, val: int) → bool:
        if val not in self.indices:
            return False
        # Swap with last element to avoid O(N) shift
        idx_to_remove = self.indices[val]
        last_val = self.values[-1]

        self.values[idx_to_remove] = last_val
        self.indices[last_val] = idx_to_remove

        # Evict last element
        self.values.pop()
        del self.indices[val]
        return True

    def getRandom(self) → int:
        return random.choice(self.values)

# Verification
rs = RandomizedSet()
assert rs.insert(10) == True
assert rs.insert(20) == True
assert rs.insert(10) == False # Already exists
assert rs.remove(10) == True
assert rs.getRandom() == 20

```

11. Follow-up Questions from Interviewer with Complete Answers

Follow-up 1

Interviewer: "In the $O(1)$ `RandomizedSet`, why is a hash map alone mathematically insufficient to achieve uniform random sampling in $O(1)$ time?"

Complete Answer: - **Non-Contiguous Bucket Layout:** - A hash map's internal buffer contains allocated capacity (e.g., 64 buckets) where only a fraction of buckets contain active keys, and others contain `EMPTY` or `DUMMY` /tombstone markers. - If you generate a random integer `rand_idx = random(0, capacity - 1)` : - If `bucket[rand_idx]` is empty or a tombstone, you must probe linearly or re-roll until a valid key is found (rejection sampling). - As the load factor drops (e.g., after many deletions where tombstones dominate), the expected number of re-rolls diverges, destroying the deterministic $O(1)$ bound. - Converting hash map keys to an array (`List(map.keys())`) takes **$O(N)$ time and $O(N)$ auxiliary**

memory. - **The Invariant:** Uniform random selection in guaranteed $O(1)$ time mathematically requires a **dense, contiguous memory array of size K** where every index $0 \leq \text{idx} < K$ maps to an active element with probability exactly $1 / K$.

Follow-up 2

Interviewer: "How does Java 8's HashMap handle high collision buckets differently than Python's open-addressing dictionary? What are the security and performance implications?"

Complete Answer: 1. **Collision Resolution Mechanisms:** - **Java 8 (Separate Chaining with Treeification):** - Each hash table bucket starts as a singly linked list (`Node<K,V>`). - If multiple keys collide in the same bucket, entries are appended to the linked list. - **Treeification:** When the number of collisions in a single bucket reaches **8** (`TREEIFY_THRESHOLD`) and total table capacity is at least 64, Java converts that linked list into a balanced **Red-Black Tree** (`TreeNode<K,V>`)! - When bucket entries drop to 6 (`UNTREEIFY_THRESHOLD`), it converts back to a linked list. - **Worst-Case Lookup Time:** Guaranteed **$O(\log N)$** even if an attacker attempts a HashDoS collision attack! - **Python 3.7+ (Compact Open Addressing):** - Python does not use separate chaining or trees. All entries reside directly in a dense array. - In the event of a collision, Python computes secondary probe locations using a pseudo-random perturbation formula: `probe = ((5 * probe) + 1 + (hash >> 5)) mod capacity`. - **Worst-Case Lookup Time:** If all keys collide, lookup degrades to **$O(N)$** . 2. **Security Implications (HashDoS Defense):** - Because Python's worst-case is $O(N)$, Python relies on **hash seed randomization** (`PYTHONHASHSEED`). At startup, CPython initializes a random 128-bit secret seed for SipHash-13/24, making it mathematically impossible for an attacker to precompute colliding strings. - Java provides defensive algorithmic resilience via Red-Black Trees ($O(\log N)$), so even if collisions are forced, the JVM CPU does not saturate.

13. Practical Exercise

Implement **Group Anagrams** without sorting strings (using a 26-element tuple of character frequencies as the hash key) to achieve $O(N * K)$ time complexity where N is word count and K is max word length.

14. Quick Revision

- Hash map lookup/insert is $O(1)$ average, $O(N)$ worst-case during severe collisions.
- Subarray sum problems with negative numbers require Prefix Sum + Hash Map.
- Keys must be immutable; mutable keys corrupt bucket invariants.
- Python uses open addressing; Java uses separate chaining (converting linked list to red-black tree when bucket size exceeds 8).

15. Interview Checklist

- [] Remembers `prefix_counts[0] = 1` for prefix sum hashing problems.

- [] Understands why sliding window fails when negative numbers are present.
- [] Can explain how to combine an array and a hash map for $O(1)$ random access deletion.
- [] Explains hash seed randomization to mitigate HashDoS.

CHAPTER 03 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Understanding of key-value associations.
- Basic knowledge of hash functions and bucket indexing.
- Scanning the relational transactions table each time would overwhelm the database with disk I/O.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 04

Two Pointers Algorithmic Patterns

PRIORITY

● High Priority

DIFFICULTY

Intermediate to Advanced

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Two Pointers Algorithmic Patterns

1. Why This Matters

Two-pointer techniques are fundamental to passing algorithmic screening interviews at top engineering hubs like IDFC FIRST Bank. The pattern eliminates nested loops ($O(N^2)$) and reduces problems to linear $O(N)$ scans with $O(1)$ auxiliary space. Interviewers frequently test two pointers on financial series, sorted settlement logs, deduplication, and pair-matching scenarios.

2. Prerequisites

- Zero-indexed array addressing.
- Sorting fundamentals (understanding that pre-sorting an array takes $O(N \log N)$ and often unlocks $O(N)$ two-pointer traversal).

3. Concept

Taxonomy of Two Pointers

1. **Opposite-Direction Pointers (Converging):** One pointer starts at the beginning (`left = 0`) and the other at the end (`right = len - 1`). They move toward each other based on comparison conditions (e.g., Two Sum on sorted array, 3-Sum, Container With Most Water).
2. **Same-Direction Pointers (Fast & Slow):** Both pointers start at index 0, but advance at different rates or under different conditions (e.g., Linked List cycle detection, in-place duplicate removal).
3. **Dual-Array Pointers:** One pointer in Array A and one pointer in Array B (e.g., Merge Sorted Arrays, intersection of two banking transaction logs).

```
flowchart LR
  subgraph Converging["Opposite-Direction (Sorted Array)"]
    L["Left Pointer →"] --> E["[ 2, 5, 7, 11, 15 ]"]
    R["← Right Pointer"] --> E
  end
```

4. Simple Example: Two Sum on a Sorted Array (O(N) Time, O(1) Space)

```
def two_sum_sorted(nums: list[int], target: int) → list[int]:
    """
    Finds 1-based indices of two numbers that sum to target.
    Requires array to be pre-sorted.
    """
    left, right = 0, len(nums) - 1
    while left < right:
        current_sum = nums[left] + nums[right]
        if current_sum == target:
            return [left + 1, right + 1]
        elif current_sum < target:
            left += 1 # Need larger sum
        else:
            right -= 1 # Need smaller sum
    return []
```

5. Real-World Banking Example: Inter-Bank Netting & Settlement Matching

At end-of-day clearing, the banking switch matches credit claims against debit obligations across institutions. Given sorted lists of receivables and payables, dual pointers can reconcile mutual netting obligations in a single $O(N + M)$ pass without building multi-gigabyte hash tables in memory.

6. Code: 3-Sum (Finding Unique Triplets Summing to Zero)

Problem Statement

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$. The solution set must **not contain duplicate triplets**.

Clarifying Questions

1. Can the elements be negative? Yes.
2. Does the order of triplets in the output matter? No.
3. What should be returned if fewer than 3 elements exist? Return an empty list `[]`.

Optimized Algorithm: Sort + Converging Two Pointers

1. Sort the array in $O(N \log N)$.
2. Iterate `i` from 0 to $N - 3$. If `nums[i] > 0`, break early (three positive numbers can never sum to 0).
3. Skip duplicate values of `nums[i]` to prevent duplicate triplets.
4. For each fixed `i`, initialize `left = i + 1` and `right = N - 1`.
5. Run the standard two-pointer check for `target = -nums[i]`.
6. When a matching triplet is found, increment `left` and decrement `right`, skipping any identical adjacent values.

```
def three_sum(nums: list[int]) → list[list[int]]:
    nums.sort() # O(N log N)
    n = len(nums)
    triplets = []

    for i in range(n - 2):
        # Optimization: If the smallest number > 0, three numbers cannot sum to 0
        if nums[i] > 0:
            break

        # Avoid duplicate triplets for the first element
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left, right = i + 1, n - 1
        target = -nums[i]

        while left < right:
            two_sum = nums[left] + nums[right]
            if two_sum == target:
                triplets.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1

                # Skip duplicate elements for second and third positions
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1

            elif two_sum < target:
                left += 1
            else:
                right -= 1

        return triplets

# Test cases
print(three_sum([-1, 0, 1, 2, -1, -4])) # [[-1, -1, 2], [-1, 0, 1]]
print(three_sum([0, 1, 1]))           # []
print(three_sum([0, 0, 0]))           # [[0, 0, 0]]
```

- **Time Complexity:** $O(N^2)$ (Sorting takes $O(N \log N)$, outer loop runs N times, inner two-pointer scan takes $O(N)$).
- **Space Complexity:** $O(1)$ auxiliary space (excluding sorting stack space, typically $O(\log N)$).
- **Edge Cases:** Array with all zeroes, all positive, all negative, array with identical duplicates.

7. How It Works Internally: Invariant Preservation

The mathematical foundation of two pointers is **search space reduction**. – In an unsorted array, there are $\text{binom}\{N\}{2} = N(N-1) / 2$ possible pairs. – By sorting the array, we establish a strict monotonic invariant: if $\text{nums}[\text{left}] + \text{nums}[\text{right}] > \text{target}$, then *no other element* paired with `nums[right]` could ever equal target, because all other available elements are $\geq \text{nums}[\text{left}]$. – Therefore, `right` can be safely discarded forever in $O(1)$ step. Each comparison eliminates an entire row or column from the candidate matrix.

8. Common Mistakes

1. **Failing to Skip Duplicates in 3-Sum:** Not skipping duplicate elements after recording a triplet results in duplicate entries in the output list.
2. **Off-by-One While Skipping Duplicates:** Incrementing `left` without checking `left < right` can cause index out-of-bounds exceptions.
3. **Applying Converging Pointers to Unsorted Arrays:** Converging two pointers *only* works if the search space exhibits monotonicity (e.g., sorted array or geometric boundary).

9. Performance / Complexity Matrix

Problem	Naive Complexity	Two-Pointer Time	Auxiliary Space
Two Sum (Sorted)	$O(N^2)$	$O(N)$	$O(1)$
3-Sum	$O(N^3)$	$O(N^2)$	$O(1)$
Container With Most Water	$O(N^2)$	$O(N)$	$O(1)$
Trapping Rain Water	$O(N^2)$	$O(N)$	$O(1)$
Remove Duplicates from Sorted Array	$O(N^2)$	$O(N)$	$O(1)$

● Easy Question: Cycle Detection & Entry in Singly Linked List (Floyd's Algorithm)

Q: Explain how to detect whether a linked list contains a cycle, and mathematically derive how to locate the exact node where the cycle begins using fast and slow pointers.

Complete Answer: Floyd's Cycle Finding Algorithm uses two pointers: `slow` (moves 1 step at a time) and `fast` (moves 2 steps at a time). 1. **Cycle Detection:** If there is no cycle, `fast` reaches `None`. If there is a cycle, `fast` enters the cycle and laps `slow` from behind, closing the gap by 1 node per step until `slow == fast`. 2. **Finding the Cycle Entry Point (Mathematical Proof):** - Let `F` = distance from head to cycle entrance. - Let `C` = length of the cycle. - Let `a` = distance from cycle entrance to the meeting point. - Distance traveled by `slow` = `F + a`. - Distance traveled by `fast` = `F + a + n * C` (where `n` is number of complete loops `fast` made). - Since `fast` travels at 2x speed: $2 * (F + a) = F + a + n * C$ $F + a = n * C$ $F = n * C - a = (n - 1) * C + (C - a)$. - Notice: `C - a` is the remaining distance from the meeting point to the cycle entrance! - Therefore, if we place one pointer at the `head` and keep another pointer at the `meeting_point`, and advance both at 1 step per tick, they will collide at the cycle entrance after exactly `F` steps.

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def detect_cycle_entry(head: ListNode | None) → ListNode | None:
    """Detects cycle and returns the entrance node.
    Time Complexity: O(N).
    Auxiliary Space: O(1).
    """
    if not head or not head.next:
        return None

    slow = head
    fast = head

    # Step 1: Detect collision
    has_cycle = False
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            has_cycle = True
            break

    if not has_cycle:
        return None

    # Step 2: Reset slow to head; advance both at 1x speed to find entry
    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next

    return slow

```

● Medium Question: Container With Most Water

Q: Given an array representing vertical lines, find two lines that together with the x-axis form a container containing the most water.

Complete Answer: Use converging two pointers starting at opposite boundaries (`left = 0` , `right = len(height) - 1`). Compute `current_area = min(height[left], height[right]) * (right - left)` . Advance the pointer with the smaller height.

```
def max_area(height: list[int]) → int:
    """Finds maximum container water volume.
    Time Complexity: O(N) single pass.
    Auxiliary Space: O(1).
    """
    left, right = 0, len(height) - 1
    max_water = 0

    while left < right:
        width = right - left
        h = min(height[left], height[right])
        max_water = max(max_water, h * width)

        # Greedily discard the limiting bottleneck
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_water

# Verification
assert max_area([1, 8, 6, 2, 5, 4, 8, 3, 7]) == 49
assert max_area([1, 1]) == 1
```

● Hard Question: Trapping Rain Water in O(1) Auxiliary Space

Q: Given an array of non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining using strictly **O(1) auxiliary space**.

Complete Answer: - **Core Principle:** Water trapped at index `i` is determined by `min(max_left, max_right) - height[i]`. - **Two-Pointer Optimization:** Instead of precomputing left and right max arrays (which takes O(N) auxiliary memory), use two pointers (`left` and `right`) and maintain `left_max` and `right_max`. - If `left_max < right_max`, the water capacity on the left side is strictly constrained by `left_max` regardless of future peaks on the right. We process `height[left]` and advance `left`. - Otherwise, water on the right side is constrained by `right_max`. We process `height[right]` and advance `right`.

```
def trap_rain_water(height: list[int]) → int:
    """Calculates trapped rainwater in O(N) time and O(1) space."""
    if not height:
        return 0

    left, right = 0, len(height) - 1
    left_max = height[left]
    right_max = height[right]
    total_water = 0

    while left < right:
        if left_max < right_max:
            left += 1
            left_max = max(left_max, height[left])
            total_water += left_max - height[left]
        else:
            right -= 1
            right_max = max(right_max, height[right])
            total_water += right_max - height[right]

    return total_water

# Verification
assert trap_rain_water([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]) == 6
assert trap_rain_water([4, 2, 0, 3, 2, 5]) == 9
```

11. Follow-up Questions from Interviewer with Complete Answers

Follow-up 1

Interviewer: "In Container With Most Water, why is it mathematically safe to advance the pointer pointing to the shorter line? Could advancing the taller line ever yield a larger area?"

Complete Answer: - The area between lines L and R is: $\text{Area}(L, R) = \min(H[L], H[R]) * (R - L)$. - Suppose without loss of generality that $H[L] \leq H[R]$. Then $\text{Area}(L, R) = H[L] * (R - L)$. - If we keep L fixed and move the taller line R inwards to any index $R' < R$: - The width strictly shrinks: $(R' - L) < (R - L)$. - The new height is $\min(H[L], H[R']) \leq H[L]$. - Therefore, $\text{Area}(L, R') \leq H[L] * (R' - L) < H[L] * (R - L) = \text{Area}(L, R)$. - **Mathematical Conclusion:** For the current L , pairing it with *any* other inner line will strictly produce a smaller area than $\text{Area}(L, R)$. - Therefore, line L can never be part of any larger container than the one we have already examined. It is mathematically impossible to miss the global maximum by discarding L ($\text{left} += 1$).

Follow-up 2

Interviewer: "Can you generalize 3-Sum to K-Sum? How do you implement a clean recursive reduction in Python, and what is the tight asymptotic bound?"

Complete Answer: - **Generalized K-Sum Reduction Strategy:** 1. Sort the input array once in $O(N \log N)$. 2. Reduce $K\text{-Sum}$ recursively to $(K - 1)\text{-Sum}$ by fixing the first element and calling $\text{K_sum}(\text{target} - \text{nums}[i])$. 3. Base Case: When $K = 2$, solve using the standard two-pointer technique in $O(N)$ time. 4. **Pruning Optimizations:** - If $\text{nums}[\text{start}] * K > \text{target}$ or $\text{nums}[-1] * K < \text{target}$, break

immediately (impossible to reach target). - Skip identical consecutive numbers (`if i > start and nums[i] == nums[i - 1]: continue`) to guarantee uniqueness. - **Time Complexity:** $O(N^{(K - 1)})$ time.
 For 3-Sum: $O(N^2)$. For 4-Sum: $O(N^3)$.

```
def k_sum(nums: list[int], target: int, k: int) → list[list[int]]:
    """Generalized K-Sum solver with recursion and duplicate skipping."""
    nums.sort()

    def k_sum_helper(start: int, target: int, k: int) → list[list[int]]:
        res = []
        if start ≥ len(nums):
            return res

        # Average value pruning
        avg = target // k
        if nums[start] > avg or nums[-1] < avg:
            return res

        # Base case: 2-Sum via two pointers
        if k == 2:
            left, right = start, len(nums) - 1
            while left < right:
                curr_sum = nums[left] + nums[right]
                if curr_sum == target:
                    res.append([nums[left], nums[right]])
                    left += 1
                    right -= 1
                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1
                elif curr_sum < target:
                    left += 1
                else:
                    right -= 1
            return res

        # Recursive step: reduce K to K-1
        for i in range(start, len(nums) - k + 1):
            if i > start and nums[i] == nums[i - 1]:
                continue
            sub_results = k_sum_helper(i + 1, target - nums[i], k - 1)
            for sub in sub_results:
                res.append([nums[i]] + sub)

        return res

    return k_sum_helper(0, target, k)

# Verification
assert k_sum([1, 0, -1, 0, -2, 2], 0, 4) == [
    [-2, -1, 1, 2],
    [-2, 0, 0, 2],
    [-1, 0, 0, 1]
]
```

13. Practical Exercise

Implement **Trapping Rain Water** using two pointers: Maintain `left_max` and `right_max`. If `height[left] < height[right]`, process `left`: if `height[left] ≥ left_max`, update `left_max`; otherwise, add `left_max - height[left]` to total water. Then increment `left`. Handle `right` symmetrically.

14. Quick Revision

- Converging pointers require monotonic sorted arrays.
- Fast and slow pointers find cycle lengths and midpoints of linked lists.
- 3-Sum = Sort ($O(N \log N)$) + Outer loop (N) + Two pointers (N) = $O(N^2)$.
- Always advance the bottleneck pointer in area maximization problems.

15. Interview Checklist

- [] Writes 3-Sum cleanly without duplicate triplets.
- [] Proves mathematically why the shorter line moves in Container With Most Water.
- [] Solves Trapping Rain Water in $O(N)$ time and $O(1)$ auxiliary space.
- [] Can explain fast and slow pointer cycle detection.



CHAPTER 04 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Zero-indexed array addressing.
- Sorting fundamentals (understanding that pre-sorting an array takes $O(N \log N)$ and often unlocks $O(N)$ two-pointer traversal)
- Time Complexity: $O(N^2)$ (Sorting takes $O(N \log N)$, outer loop runs N times, inner two-pointer scan takes $O(N)$).



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 05

Sliding Window Algorithmic Patterns

PRIORITY

● High Priority

DIFFICULTY

Intermediate to Advanced

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Sliding Window Algorithmic Patterns

1. Why This Matters

Sliding window algorithms are among the most frequently tested algorithmic techniques at IDFC FIRST Bank and across the FinTech industry. They appear in problems involving contiguous subarrays, string substrings, rolling rate limiters, burst traffic detection, and streaming transaction analytics. The sliding window reduces $O(N^2)$ brute-force window evaluations into optimal $O(N)$ linear time.

2. Prerequisites

- Two-pointer basics.
- Hash map and hash set frequency tracking.

3. Concept

Types of Sliding Windows

1. **Fixed-Size Window:** The window length K is constant. As the right pointer advances, the left pointer advances simultaneously (`left = right - K + 1`), adding the incoming element and removing the outgoing element in $O(1)$.
2. **Dynamic / Variable-Size Window:** The window expands by advancing the `right` pointer to satisfy an invariant, and contracts by advancing the `left` pointer when the invariant is violated (e.g., finding the shortest or longest subarray meeting a condition).

```
flowchart LR
    subgraph Window ["Dynamic Window Expansion & Contraction"]
        direction TB
        w1["1. Expand: right += 1 (Add nums[right] to window state)"]
        w2["2. Check: Is window invalid?"]
        w3["3. Shrink: while invalid: left += 1 (Remove nums[left])"]
        w4["4. Record: Update global answer (max_len or min_len)"]
        w1 --> w2 --> w3 --> w4
    end
    end
```

4. Simple Example: Maximum Sum Subarray of Fixed Size K

```
def max_sub_array_fixed(nums: list[int], k: int) → int:
    """
    Finds maximum sum of any contiguous subarray of size k.
    Complexity: O(N) Time, O(1) Space.
    """
    if len(nums) < k:
        return 0

    # Initial window sum
    window_sum = sum(nums[:k])
    max_sum = window_sum

    # Slide window
    for right in range(k, len(nums)):
        window_sum += nums[right] - nums[right - k] # Add incoming, subtract outgoing
        max_sum = max(max_sum, window_sum)

    return max_sum
```

5. Real-World Banking Example: Rolling Rate Limiter & Transaction Spikes

To prevent card testing attacks, an API gateway monitors incoming payment attempts: "Has a merchant terminal initiated more than 100 requests in any rolling 60-second window?" - A sliding window tracks request timestamps. As time moves forward, timestamps older than 60 seconds expire from the left edge of the window, maintaining continuous real-time throughput metrics without resetting on artificial minute boundaries.

6. Code: Longest Substring Without Repeating Characters

Problem Statement

Given a string `s`, find the length of the **longest substring** without repeating characters.

Clarifying Questions

1. What characters can the string contain? English letters, digits, symbols, and spaces.
2. Can the input be empty? Yes, empty string returns 0.

Brute Force Approach

Check every possible substring ($O(N^2)$), and for each check whether all characters are unique using a set ($O(N)$). - **Time Complexity:** $O(N^3)$ - **Space Complexity:** $O(\min(N, M))$ where M is alphabet size.

Optimized Approach: Dynamic Sliding Window with Last Seen Index Map

Instead of shrinking the window one character at a time, store the **most recent index** where each character was seen. When a duplicate character is encountered, jump `left` directly past the previous instance of that character: `left = max(left, last_seen[char] + 1)`

```
def length_of_longest_substring(s: str) → int:
    last_seen = {} # char → index
    left = 0
    max_len = 0

    for right, char in enumerate(s):
        # If char was seen and is within the current window
        if char in last_seen and last_seen[char] ≥ left:
            left = last_seen[char] + 1

        last_seen[char] = right
        max_len = max(max_len, right - left + 1)

    return max_len

# Test cases
print(length_of_longest_substring("abcabcbb")) # 3 ("abc")
print(length_of_longest_substring("bbbbbb")) # 1 ("b")
print(length_of_longest_substring("pwwkew")) # 3 ("wke")
print(length_of_longest_substring("")) # 0
```

- **Time Complexity:** $O(N)$ (Each character is processed at most once by `right`).
- **Space Complexity:** $O(\min(N, M))$ where M is the character set size (e.g., 128 for ASCII).
- **Edge Cases:** Empty string, string with all unique characters, string with identical characters, strings with spaces and punctuation.

7. How It Works Internally: Amortized Analysis of Sliding Windows

Junior engineers often mistakenly evaluate dynamic sliding windows as $O(N^2)$ because they see a `while` loop nested inside a `for` loop:

```
for right in range(n):
    while window_is_invalid():
        left += 1
```

Why it is strictly $O(N)$ Amortized: – The `right` pointer increments exactly N times. – The `left` pointer only ever moves forward; it *never* decrements or resets. – Therefore, across the entire execution of the algorithm, `left` increments at most N times. – Total pointer operations = $N + N = 2N \Rightarrow O(N)$ time.

8. Common Mistakes

1. **Forgetting `last_seen[char] ≥ left`:** In the index jump optimization, if a character was seen at index 1, but `left` has already advanced to index 5, forgetting this check pulls `left` *backwards*, corrupting the window!
2. **Incorrect Window Size Calculation:** Window length is always `right - left + 1` (inclusive), not `right - left`.
3. **Attempting Sliding Window on Negative Number Arrays:** Finding a subarray summing to K cannot use a sliding window when negative numbers are present, because expanding the window can *decrease* the sum.

9. Performance / Complexity Matrix

Problem	Naive Time	Sliding Window Time	Auxiliary Space
Max Sum Subarray of Size K	$O(N * K)$	$O(N)$	$O(1)$
Longest Substring Without Repeating Characters	$O(N^3)$	$O(N)$	$O(M)$
Minimum Size Subarray Sum	$O(N^2)$	$O(N)$	$O(1)$
Sliding Window Maximum	$O(N * K)$	$O(N)$ (Monotonic Deque)	$O(K)$

● Easy Question: Fixed-Size Window Running Sum

Q: Explain how a fixed-size sliding window of size `K` calculates the maximum sum of any contiguous subarray in $O(N)$ time and $O(1)$ auxiliary space.

Complete Answer: Rather than re-summing `K` elements in $O(K)$ at every step (which would be $O(N * K)$), compute the sum of the first `K` elements once. Then slide the window from `i = K` to `N - 1`: add the incoming element `arr[i]` and subtract the outgoing element `arr[i - K]`. Each update is strictly $O(1)$.

```
def max_sub_array_of_size_k(k: int, arr: list[int]) → int:
    """Computes maximum subarray sum of fixed window size k.
    Time Complexity: O(N).
    Auxiliary Space: O(1).
    """
    if len(arr) < k:
        return 0

    window_sum = sum(arr[:k])
    max_sum = window_sum

    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i - k] # O(1) state delta
        max_sum = max(max_sum, window_sum)

    return max_sum

# Verification
assert max_sub_array_of_size_k(3, [2, 1, 5, 1, 3, 2]) == 9 # [5, 1, 3] = 9
```

● Medium Question: Minimum Size Subarray Sum

Q: Given an array of positive integers `nums` and a positive integer `target`, find the minimal length of a contiguous subarray of which the sum is greater than or equal to `target`. If there is no such subarray, return 0.

Complete Answer: Because all numbers in `nums` are strictly positive, the cumulative sum within a window increases monotonically as we expand `right`, and decreases monotonically as we shrink `left`. 1. Expand window: add `nums[right]` to `current_sum`. 2. While `current_sum ≥ target`, record

candidate length `right - left + 1`, subtract `nums[left]`, and increment `left`. 3. Each index is visited at most twice (once by `right`, once by `left`).

```
def min_sub_array_len(target: int, nums: list[int]) -> int:
    """Finds minimum length subarray with sum ≥ target.
    Time Complexity: O(N) amortized.
    Auxiliary Space: O(1).
    """
    left = 0
    current_sum = 0
    min_len = float("inf")

    for right in range(len(nums)):
        current_sum += nums[right]

        while current_sum ≥ target:
            min_len = min(min_len, right - left + 1)
            current_sum -= nums[left]
            left += 1

    return min_len if min_len ≠ float("inf") else 0

# Verification
assert min_sub_array_len(7, [2, 3, 1, 2, 4, 3]) == 2 # [4, 3] length 2
assert min_sub_array_len(11, [1, 1, 1, 1, 1]) == 0
```

● Hard Question: Sliding Window Maximum via Monotonic Deque

Q: Given an integer array `nums` and a sliding window of size `k` moving from left to right, return the maximum element in each window. Solve in strictly **O(N) time** and **O(k) space**.

Complete Answer: A naive scan takes `O(N * k)`. A max-heap takes `O(N log k)`. To achieve strictly `O(N)`, use a **Monotonic Deque** storing array *indices*: - Maintain elements in strictly decreasing order of their values (`deque[0]` is always the largest element in the current window). - For each new element `nums[i]`: 1. Remove expired elements from the front: `while q and q[0] ≤ i - k: q.popleft()`. 2. Maintain monotonic invariant from the back: `while q and nums[q[-1]] ≤ nums[i]: q.pop()`. 3. Append `i` to the back. 4. Once `i ≥ k - 1`, the maximum of the current window is `nums[q[0]]`.

```

from collections import deque

def max_sliding_window(nums: list[int], k: int) → list[int]:
    """Finds maximum value in each sliding window of size k in O(N) time."""
    q = deque() # Stores indices
    result = []

    for i in range(len(nums)):
        # 1. Remove indices outside current window
        if q and q[0] ≤ i - k:
            q.popleft()

        # 2. Maintain decreasing monotonic order
        while q and nums[q[-1]] ≤ nums[i]:
            q.pop()

        # 3. Add current element index
        q.append(i)

        # 4. Record current window maximum
        if i ≥ k - 1:
            result.append(nums[q[0]])

    return result

# Verification
assert max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3) == [3, 3, 5, 5, 6, 7]
assert max_sliding_window([1], 1) == [1]

```

11. Follow-up Questions from Interviewer with Complete Answers

Follow-up 1

Interviewer: "In the Monotonic Deque solution for Sliding Window Maximum, why do we store indices in the deque rather than values?"

Complete Answer: – Boundary Verification: If we stored values in the deque, we would have no way to determine whether the front element (`deque[0]`) has physically fallen outside the sliding window when duplicate values exist in the array. – **Example:** Suppose `nums = [3, 3, 3, 1]`, `k = 2`. – Storing values: we cannot differentiate whether the `3` at the front of the deque belongs to index 0 (which has expired at `i = 2`) or index 1 (which is still active). – Storing indices: the condition `q[0] ≤ i - k` deterministically checks whether the element's index has expired. We access the actual value in O(1) via `nums[q[0]]`. – **Amortized Proof:** Every index enters the deque exactly once and is evicted at most once (either from the front when it expires, or from the back when surpassed by a larger number). Total operations across N elements are bounded by `2N = O(N)`.

Follow-up 2

Interviewer: "How does a Leaky Bucket rate limiter differ from a Sliding Window Log rate limiter in banking payment gateways? How do you implement Sliding Window Log using Redis?"

Complete Answer: 1. **Algorithmic Comparison:** – **Leaky Bucket:** Requests enter a buffer and leak out at a strictly constant rate (e.g., exactly 100 txns/sec to the core banking switch). Smooths out bursty traffic, but adds queuing latency during bursts. – **Token Bucket:** Tokens are replenished at a constant

rate up to capacity. Allows immediate processing of short traffic bursts up to the token limit. - **Sliding Window Log**: Records each transaction timestamp in a sliding time window (e.g., last 60 seconds). Provides exact mathematical rate limiting without boundary-edge burst vulnerabilities (unlike Fixed Window Counter). 2. **Redis Implementation of Sliding Window Log**: - Use a **Redis Sorted Set (ZSET)** where `key = f"rate_limit:{vpa_id}"`, `score = current_unix_timestamp_ms`, and `member = transaction_uuid`. - In a single atomic Lua script: 1. Remove timestamps older than `now - window_size_ms`: `ZREMRANGEBYSCORE key 0 (now - window_size_ms)`. 2. Count current elements in window: `current_count = ZCARD key`. 3. If `current_count < limit`: - `ZADD key now uuid`. - `EXPIRE key window_seconds`. - Return 1 (Allow). 4. Else: - Return 0 (Block / HTTP 429 Too Many Requests).

```
import time

def simulate_sliding_window_rate_limiter(requests_log: list[float], limit: int, window_sec: float) → list[bool]:
    """Simulates sliding window log rate limiter."""
    allowed = []
    active_window = []

    for req_time in requests_log:
        # Evict timestamps older than window boundary
        cutoff = req_time - window_sec
        active_window = [t for t in active_window if t > cutoff]

        if len(active_window) < limit:
            active_window.append(req_time)
            allowed.append(True)
        else:
            allowed.append(False)

    return allowed

# Verification: Limit 3 requests per 5.0 seconds
req_times = [1.0, 2.0, 3.0, 4.0, 7.0] # at t=4.0, 4th request within 5s is rejected
assert simulate_sliding_window_rate_limiter(req_times, 3, 5.0) == [True, True, True, False, True]
```

13. Practical Exercise

Implement **Sliding Window Maximum** in O(N) using Python's `collections.deque`: Maintain a deque of indices where values are strictly in descending order. Before appending `right`, pop all smaller elements from the back of the deque. Pop the front element if its index is $\leq \text{right} - k$.

14. Quick Revision

- Fixed window: `window_sum += incoming - outgoing`.
- Dynamic window: expand with `right`, shrink with `left` while condition holds.
- Complexity is O(N) amortized because each element is visited at most twice.
- Window size = `right - left + 1`.
- Negative numbers invalidate sliding window monotonicity; use Prefix Sum + Hash Map instead.

15. Interview Checklist

- [] Writes Longest Substring Without Repeating Characters with index jumps.
- [] Proves $O(N)$ amortized complexity for nested while loop.
- [] Remembers that sliding window requires positive numbers for sum thresholds.
- [] Understands Monotonic Deque for Sliding Window Maximum.



CHAPTER 05 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Two-pointer basics.
- Hash map and hash set frequency tracking.
- A sliding window tracks request timestamps. As time moves forward, timestamps older than 60 seconds expire from the left



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 06

Python for Coding Interviews & DSA Mastery

PRIORITY

● High Priority

DIFFICULTY

Intermediate to Advanced

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Python for Coding Interviews & DSA Mastery

1. Why This Matters

Coding interviews mein speed aur clean code sabse zyada count hota hai. Python ki standard library mein aisi built-in data structures hain (`deque` , `heapq` , `Counter` , `defaultdict` , `bisect`) jo aapko 50 lines ka boilerplate likhne ke bajaye 2 lines mein optimal logic implement karne deti hain. Lekin agar aapko inka internal time complexity aur edge cases nahi pata (jaise `list.pop(0)` ka $O(N)$ hona), toh interviewer aapko complexity round mein disqualify kar dega.

2. Python DSA Decision Tree: "Kab Kya Use Karein?"

```
flowchart TD
    Problem["Algorithmic Requirement"] --> R1["Unique elements or lookup in O(1)?"]
    R1 -->|Yes| S["Use set or dict"]
    R1 -->|No| R2["FIFO Queue or BFS?"]
    R2 -->|Yes| DQ["Use collections.deque (popleft O(1))"]
    R2 -->|No| R3["Top K elements / Min / Max in O(log N)?"]
    R3 -->|Yes| HQ["Use heapq (Min-Heap / Negated Max-Heap)"]
    R3 -->|No| R4["Frequency counts or grouped lists?"]
    R4 -->|Yes| C["Use collections.Counter or defaultdict"]
    R4 -->|No| R5["Binary Search on sorted data?"]
    R5 -->|Yes| B["Use bisect.bisect_left / bisect_right"]
    R5 -->|No| L["Use standard list"]
```

3. Data Structure Deep-Dive & Complexity Matrix

1. `list` — Dynamic Array

- **When to use:** Index-based access, sequential scans, stacks (LIFO via `append()` and `pop()`).
- **Under the hood:** Contiguous array of pointers (`PyObject**`). Over-allocates capacity geometrically.

Method	Time Complexity	Notes & Gotchas
<code>arr[i]</code>	$O(1)$	Direct pointer arithmetic in memory
<code>arr.append(x)</code>	$O(1)$ amortized	Geometric resizing
<code>arr.pop()</code>	$O(1)$	Pops from end (Stack LIFO)
<code>arr.pop(0)</code>	$O(N)$ (DANGEROUS)	Shifts all $N-1$ pointers left in RAM!
<code>arr.insert(0, x)</code>	$O(N)$	Shifts all elements right
<code>x in arr</code>	$O(N)$	Linear scan

2. `collections.deque` — Double-Ended Queue

- **When to use:** BFS (Breadth-First Search), Sliding Window Maximum, FIFO queues.
- **Under the hood:** Doubly-linked list of 64-element block buffers.

```
from collections import deque

queue = deque([10, 20, 30])
queue.append(40)      #  $O(1)$  Push Right
queue.appendleft(0)   #  $O(1)$  Push Left
first = queue.popleft() #  $O(1)$  Pop Left (NEVER use list.pop(0)!)
last = queue.pop()    #  $O(1)$  Pop Right
```

3. `heapq` — Priority Queue (Min-Heap)

- **When to use:** "Top K frequent elements", "Kth largest element", "Median of a data stream", Dijkstra's algorithm.
- **Under the hood:** Binary Min-Heap implemented over a standard Python list. Smallest element is always at `heap[0]`.

```
import heapq

# Min-Heap (Default)
min_heap = []
heapq.heappush(min_heap, 25) #  $O(\log N)$ 
heapq.heappush(min_heap, 5)
heapq.heappush(min_heap, 50)
smallest = heapq.heappop(min_heap) # 5 in  $O(\log N)$ 

# Max-Heap Pattern: Negate values upon push and pop!
max_heap = []
heapq.heappush(max_heap, -25)
heapq.heappush(max_heap, -5)
heapq.heappush(max_heap, -50)
largest = -heapq.heappop(max_heap) # 50

# In-Place Heapify: Converts an existing list into a heap in  $O(N)$  linear time!
nums = [15, 3, 20, 1, 9]
heapq.heapify(nums) #  $O(N)$  time (Faster than pushing  $N$  times  $O(N \log N)$ !)
```

4. `collections.Counter` & `collections.defaultdict`

- **When to use:** Character counts, anagrams, grouping items by key without `KeyError`.

```
from collections import Counter, defaultdict

# 1. Counter: Instant frequency map
counts = Counter("idfcfirstbank")
print(counts["i"]) # 2
print(counts.most_common(2)) # [('i', 2), ('f', 1)]

# 2. defaultdict: Eliminates 'if key not in dict' boilerplate
# Graph adjacency list representation:
adj_list = defaultdict(list)
edges = [(1, 2), (1, 3), (2, 4)]
for u, v in edges:
    adj_list[u].append(v)
# adj_list[1] → [2, 3]; adj_list[5] → [] (No KeyError!)
```

5. `bisect` — Binary Search Module

- **When to use:** Finding insertion point in sorted arrays, range query boundaries in $O(\log N)$.

```
import bisect

sorted_rates = [3.5, 5.0, 7.25, 8.5, 9.1]

# bisect_left: First index where val can be inserted maintaining sort order (≥ val)
idx_left = bisect.bisect_left(sorted_rates, 7.25) # Index 2

# bisect_right: Index after any existing equal elements (> val)
idx_right = bisect.bisect_right(sorted_rates, 7.25) # Index 3
```

4. Custom Sorting & Comparator Keys

In technical interviews, sorting complex objects or breaking ties is very common:

```
# Problem: Sort transactions by:
# 1. Status: 'FAILED' first, then 'PENDING', then 'SUCCESS'
# 2. Amount: Descending (Higher amounts first)
# 3. Reference ID: Ascending (Lexicographical)

transactions = [
    {"ref": "TX1", "amount": 5000, "status": "SUCCESS"},
    {"ref": "TX2", "amount": 12000, "status": "FAILED"},
    {"ref": "TX3", "amount": 8000, "status": "FAILED"},
    {"ref": "TX4", "amount": 5000, "status": "SUCCESS"},
]

status_priority = {"FAILED": 0, "PENDING": 1, "SUCCESS": 2}

# Custom key function returning a tuple:
sorted_txns = sorted(
    transactions,
    key=lambda x: (status_priority[x["status"]], -x["amount"], x["ref"])
)
# Result: TX2 (12k FAILED), then TX3 (8k FAILED), then TX1 (5k SUCCESS), then TX4
```

5. Recursion, BFS & DFS Templates in Python

1. BFS Template (Iterative with `deque`)

```
def bfs_shortest_path(graph, start, target):
    queue = deque([(start, 0)]) # (node, distance)
    visited = {start}

    while queue:
        node, dist = queue.popleft() # 0(1)
        if node == target:
            return dist

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))

    return -1 # Target unreachable
```

2. DFS Template (Recursive with Visited Set)

```
def dfs_traversal(graph, node, visited=None):
    if visited is None:
        visited = set()

    visited.add(node)
    # Process current node
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs_traversal(graph, neighbor, visited)

    return visited
```

6. Top 5 Python DSA Interview Traps

1. **Using `list.pop(0)` in BFS:** Turns $O(V + E)$ into quadratic $O(V^2)$ because each pop shifts entire list memory. Always use `collections.deque.popleft()`.
2. **Mutating list while iterating:** `for x in arr: if condition: arr.remove(x)` skips elements because indices shift dynamically. Create a new list via comprehension or iterate backwards.
3. **Using 2D lists with multiplication:** `matrix = [[0] * 3] * 3` creates 3 references to the **same single row**! Modifying `matrix[0][0] = 1` mutates all 3 rows. Always use: `matrix = [[0] * cols for _ in range(rows)]`.
4. **Neglecting recursion depth limit:** Python default recursion depth is 1,000. In deep graph/tree problems, `sys.setrecursionlimit(200000)` is required or convert recursion to iterative stack.
5. **Float comparison precision:** `0.1 + 0.2 == 0.3` evaluates to `False`. Use `math.isclose(a, b)` for floating-point equality.



CHAPTER 06 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- When to use: Index-based access, sequential scans, stacks (LIFO via `append()` and `pop()`).
- Under the hood: Contiguous array of pointers (`PyObject``). Over-allocates capacity geometrically.
- When to use: BFS (Breadth-First Search), Sliding Window Maximum, FIFO queues.



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 07

DSA Last–Minute Algorithmic Patterns Cheatsheet

PRIORITY

● High Priority

DIFFICULTY

Intermediate to Advanced

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

DSA Last–Minute Algorithmic Patterns Cheatsheet

The Top 14 Algorithmic Patterns Quick Reference

Pattern	Trigger in Problem Statement	Core Data Structure	Typical Complexity
1. Frequency Hash Map	"Find duplicates", "Frequency of elements", "Anagrams"	<code>collections.Counter</code> / <code>dict</code>	O(N) Time, O(N) Space
2. Two Pointers	"Sorted array", "Find pair summing to target", "Palindromes"	Two integer indices (<code>L</code> , <code>R</code>)	O(N) Time, O(1) Space
3. Sliding Window	"Contiguous subarray/substring", "Longest/Shortest with condition"	Two indices + Hash Map	O(N) Time, O(K) Space
4. Fast & Slow Pointers	"Cycle detection in linked list", "Find middle of linked list"	<code>slow</code> , <code>fast</code> pointers	O(N) Time, O(1) Space
5. Prefix Sum	"Subarray sum equals K", "Range sum queries"	Array / Running sum	O(N) Time, O(1) query
6. Monotonic Stack	"Next greater element", "Daily temperatures", "Stock span"	<code>list</code> as stack	O(N) Time, O(N) Space
7. Top K Elements	"Find K largest / K most frequent", "Running median"	<code>heapq</code> (Min-Heap / Max-Heap)	O(N log K) Time
8. Binary Search	"Sorted input", "Find peak", "Search rotated sorted array", "Min of Max"	<code>left</code> , <code>right</code> , <code>mid</code>	O(log N) Time, O(1) Space
9. BFS (Level Order)	"Shortest path in unweighted graph/grid", "Level-by-level tree"	<code>collections.deque</code>	O(V + E) Time, O(V) Space
10. DFS (Backtracking)	"Generate all combinations/permutations/subsets", "Islands in grid"	Recursion / Call Stack	O(2 ^N) or O(N!) Time
11. Merge Intervals	"Overlapping intervals", "Meeting rooms schedule"	Sort by <code>start_time</code>	O(N log N) Time
12. Dynamic Programming	"Overlapping subproblems", "Max/Min path", "Number of ways"	1D / 2D table or memo dict	O(N) or O(N * M)
13. Greedy	"Interval scheduling", "Jump game", "Local optimum yields global"	Priority Queue or Sort	O(N log N) Time
14. Union-Find (DSU)	"Connected components", "Detect cycle in undirected graph"	<code>parent</code> and <code>rank</code> arrays	O(alpha(N)) ≈ O(1)

Python High-Yield Syntax Snippets

```
# 1. Monotonic Deque for O(1) pops and appends:
from collections import deque
q = deque()
q.append(1)
q.popleft() # O(1) vs list.pop(0) which is O(N)!

# 2. Min-Heap & Max-Heap:
import heapq
min_h = []
heapq.heappush(min_h, 10)
val = heapq.heappop(min_h)
# Max-heap: push negative values
heapq.heappush(min_h, -10)
val = -heapq.heappop(min_h)

# 3. Safe Binary Search Mid Calculation:
mid = left + (right - left) // 2

# 4. In-Place Reversal:
def reverse_range(arr, l, r):
    while l < r:
        arr[l], arr[r] = arr[r], arr[l]
        l += 1
        r -= 1

# 5. Defaultdict without KeyError:
from collections import defaultdict
graph = defaultdict(list)
counts = defaultdict(int)
```

CHAPTER 07 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Core fundamentals and architectural mechanics of DSA Last-Minute Algorithmic Patterns Cheatsheet
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases